

SET B — Answer Key & Concepts

(CONFIDENTIAL)

Each entry explains the concept, lists the bugs and gives the corrected program.

PART A — EASY

E1. Sum of Sensor Readings (Python)

Concept explained: An *accumulator* starts at the neutral value of the operation: 0 for addition, 1 for multiplication. Then you add each element. `range(n)` covers indices 0...n-1.

Bugs: (1) `total = 1` → `0` (2) `range(1, n)` skips `a[0]` → `range(n)`

```
n = int(input())
a = list(map(int, input().split()))
total = 0
for i in range(n):
    total += a[i]
print(total)
```

E2. Peak Voltage (C++)

Concept explained: To find a maximum, start from a value that can't beat any real reading, such as `INT_MIN` or the first element. Starting at 0 fails when every reading is negative. Each reading replaces the current maximum only if it is **greater**.

Bugs: (1) `mx = 0` → `INT_MIN` (2) `x < mx` → `x > mx`

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    int mx = INT_MIN;
    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        if (x > mx) mx = x;
    }
    cout << mx << endl;
}
```

E3. Factorial (Java)

Concept explained: $n! = 1 \times 2 \times \dots \times n$, so the loop must include `n`. `int` holds values up to about 2.1×10^9 , but $13!$ is already larger. `long` (up to about 9.2×10^{18}) fits up to $20!$.

Bugs: (1) `i < n` → `i <= n` (2) `int f` → `long f`

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int n = new Scanner(System.in).nextInt();
        long f = 1;
        for (int i = 1; i <= n; i++) f *= i;
        System.out.println(f);
    }
}
```

E4. Even and Odd Counter (Python)

Concept explained: `x % 2` is the remainder after dividing by 2. A remainder of 0 means the number is even.

Bug: the condition counts odd numbers as even → `x % 2 == 0`

```
n = int(input())
a = list(map(int, input().split()))
even = odd = 0
for x in a:
    if x % 2 == 0:
        even += 1
    else:
        odd += 1
print(even, odd)
```

E5. Reverse a Signal Label (C++)

Concept explained: Two-pointer reversal swaps `s[i]` with its mirror `s[n-1-i]`, and only for the first half. If the loop goes over the whole string, every character gets swapped twice and the string comes back unchanged. `s[n]` is out of bounds.

Bugs: (1) `i < n` → `i < n / 2` (2) `s[n - i]` → `s[n - 1 - i]`

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s; cin >> s;
    int n = s.size();
    for (int i = 0; i < n / 2; i++)
        swap(s[i], s[n - 1 - i]);
    cout << s << endl;
}
```

E6. Temperature Converter (Java)

Concept explained: In Java, `int / int` is integer division, so `9 / 5` is `1`. Making one operand a double (`9.0`) gives true division (`1.8`).

Bug: `9 / 5` → `9.0 / 5`

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int c = new Scanner(System.in).nextInt();
        double f = c * (9.0 / 5) + 32;
        System.out.printf("%.1f%n", f);
    }
}
```

E7. Vowel Counter (Python)

Concept explained: String comparisons are case-sensitive, so convert to lowercase before checking. `count += 1` actually means `count = (+1)`, which assigns 1 every time. The increment operator is `+=`.

Bugs: (1) check `ch.lower()` (2) `+=` → `+=`

```
s = input()
count = 0
for ch in s:
    if ch.lower() in "aeiou":
        count += 1
print(count)
```

E8. Digit Sum (C++)

Concept explained: `n % 10` gives the last digit and `n / 10` removes it. The buggy code swaps the two operations, so `n` never reaches 0 and the loop never ends.

Bug: swap the operations: `s += n % 10; n /= 10;`

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long n; cin >> n;
    int s = 0;
    while (n > 0) {
        s += n % 10;
        n /= 10;
    }
    cout << s << endl;
}
```

E9. Prime Check (Java)

Concept explained: A prime number has no divisor between 2 and $\sqrt{n}$. Every number is divisible by 1, so the loop must start at 2. By definition, 0 and 1 aren't prime. Primes are the basis of RSA cryptography.

Bugs: (1) start at `i = 2` (2) `n < 2` is not prime

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int n = new Scanner(System.in).nextInt();
        boolean p = n >= 2;
        for (int i = 2; (long) i * i <= n; i++)
            if (n % i == 0) p = false;
        System.out.println(p ? "PRIME" : "NOT PRIME");
    }
}
```

E10. Ohm's Law Calculator (Python)

Concept explained: `input().split()` returns **strings**, so convert them with `float()` before doing arithmetic. Ohm's law gives $I = V/R$, and electrical power is $P = V \times I$ (equivalently V^2/R).

Bugs: (1) convert with `map(float, ...)` (2) $P = V * R \rightarrow P = V * I$

```
V, R = map(float, input().split())
I = V / R
P = V * I
print(f"{I:.2f} {P:.2f}")
```

E11. Count the 1-bits (C++)

Concept explained: `n % 2` is the least significant bit (LSB). Dividing by 2 shifts the number right by one bit. When the remainder is 1, the bit is set.

Bug: `n % 2 == 0` counts zeros $\rightarrow$ `n % 2 == 1`

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    int c = 0;
    while (n > 0) {
        if (n % 2 == 1) c++;
        n /= 2;
    }
    cout << c << endl;
}
```

E12. Average Marks (Java)

Concept explained: `i <= n` runs $n+1$ times and tries to read a value that doesn't exist (`NoSuchElementException`). `sum / n` is integer division, so the decimal part is lost before the result is stored in the double.

Bugs: (1) `i <= n` $\rightarrow$ `i < n` (2) `(double) sum / n`

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int sum = 0;
        for (int i = 0; i < n; i++) sum += sc.nextInt();
        double avg = (double) sum / n;
        System.out.printf("%.2f%n", avg);
    }
}
```

E13. Linear Search (Python)

Concept explained: Linear search checks every element. In the buggy code, the `else: return -1` exits on the **first** mismatch. "Not found" can only be decided after the whole loop has finished.

Bug: move `return -1` after the loop

```
n, t = map(int, input().split())
a = list(map(int, input().split()))
def find(a, t):
    for i in range(len(a)):
        if a[i] == t:
            return i
    return -1
print(find(a, t))
```

E14. GCD of Two Numbers (C++)

Concept explained: Euclid's rule is $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$, and it stops when $b = 0$. Both new values must be computed from the **old** a and b . In the buggy code, `a = b` overwrites a before `a % b` uses it, so b always becomes 0. GCD is used in RSA key generation.

Bug: use a temporary variable

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long a, b; cin >> a >> b;
    while (b != 0) {
        long long t = a % b;
        a = b;
        b = t;
    }
    cout << a << endl;
}
```

E15. Word Counter (Java)

Concept explained: `split(" ")` creates empty strings between consecutive spaces and at the start of the line. `trim()` removes the leading and trailing spaces, and the regex `\\s+` treats any run of whitespace as one separator.

Bugs: (1) `trim()` first (2) `split("\\s+")`

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        String s = new Scanner(System.in).nextLine();
        String[] w = s.trim().split("\\s+");
        System.out.println(w.length);
    }
}
```

E16. Atbash Cipher (Python)

Concept explained: Atbash is a substitution cipher that maps a letter at position p (0-based) to position $25-p$. In character codes, that's `chr(ord('a') + ord('z') - ord(c))`. The result must be built up with `+=`, otherwise it is overwritten on every iteration.

Bugs: (1) the formula is missing `ord('a') +` (2) `res =` → `res +=`

```
s = input().strip()
res = ""
for c in s:
    if c.isalpha():
        res += chr(ord('a') + ord('z') - ord(c))
    else:
        res += c
print(res)
```

E17. Binary to Decimal (C++)

Concept explained: Horner's method reads the digits from the left and computes `r = r*2 + digit`. The character `'1'` has ASCII code 49, so subtract `'0'` to get the digit 1. The loop must start at index 0, which is the most significant bit.

Bugs: (1) `i = 1` → `i = 0` (2) `s[i] - '0'`

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s; cin >> s;
    long long r = 0;
    for (int i = 0; i < (int)s.size(); i++)
        r = r * 2 + (s[i] - '0');
    cout << r << endl;
}
```

E18. Leap Year (Java)

Concept explained: `&&` binds tighter than `||`, so the buggy condition reads `y%4==0 || (y%100!=0 && y%400==0)`, which makes 1900 a leap year. Use parentheses to state the real rule.

Bug: `(y % 4 == 0 && y % 100 != 0) || y % 400 == 0`

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int y = new Scanner(System.in).nextInt();
        if ((y % 4 == 0 && y % 100 != 0) || y % 400 == 0)
            System.out.println("LEAP");
        else
            System.out.println("NOT LEAP");
    }
}
```

PART B — MODERATE

BM1. Second Highest Score (Python)

Concept explained: Keep the two best values in a single pass. When a new maximum appears, the **old** maximum becomes the second, so update `second` before `first`. Duplicates of the maximum must not count as the second. Sentinels must be $-\infty$ because the values can be negative.

Bugs: (1) initialize with `float('-inf')` (2) wrong update order (3) `elif x > second and x != first` (4) print -1 when no second exists

```
n = int(input())
a = list(map(int, input().split()))
first = second = float('-inf')
for x in a:
    if x > first:
        second = first
        first = x
    elif x > second and x != first:
        second = x
print(second if second != float('-inf') else -1)
```

BM2. Bubble Sort (C++)

Concept explained: Each pass bubbles the largest remaining element to the end, so the inner loop compares `a[j]` with `a[j+1]` for `j < n-i-1`, which keeps `j+1` in bounds. A swap needs a temporary variable. Without one, both cells end up with the same value. Stop early when a full pass makes **no** swap, because the array is already sorted.

Bugs: (1) `j < n - i - 1` (2) use `swap(a[j], a[j+1])` (3) `if (!swapped) break;`

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) {
                swap(a[j], a[j + 1]);
                swapped = true;
            }
        if (!swapped) break;
    }
    for (int i = 0; i < n; i++) cout << a[i] << " \n"[i == n - 1];
}

```

BM3. Anagram Checker (Java)

Concept explained: Two words are anagrams if every letter appears the same number of times in both. Count the letters of the first word up and the second word down, and every counter should end at 0. The alphabet has 26 letters, so with 25 slots 'z' overflows the array. When a check fails, the loop must not reset the flag to true.

Bugs: (1) `new int[26]` (2) the second loop must use `cnt[c - 'a']--` (3) remove the `else ok = true`

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String a = sc.next(), b = sc.next();
        int[] cnt = new int[26];
        for (char c : a.toCharArray()) cnt[c - 'a']++;
        for (char c : b.toCharArray()) cnt[c - 'a']--;
        boolean ok = true;
        for (int x : cnt)
            if (x != 0) ok = false;
        System.out.println(ok ? "YES" : "NO");
    }
}

```

BM4. Run-Length Encoding (Python)

Concept explained: Run-length encoding (RLE) is a simple lossless compression scheme, used for example in fax machines and bitmap formats. Each run starts with a count of 1, because the current character itself counts. The final run is never closed by a change of character, so it must be appended after the loop.

Bugs: (1) `count = 1` initially (2) reset to 1 (3) append the last group after the loop

```

s = input().strip()
res = ""
count = 1
for i in range(1, len(s)):
    if s[i] == s[i - 1]:
        count += 1
    else:
        res += s[i - 1] + str(count)
        count = 1
res += s[-1] + str(count)
print(res)

```

PART C — HARD

BH1. Counting Chip Islands (C++)

Concept explained: This is connected-component counting with BFS. The direction arrays (dx, dy) must describe down, up, right and **left**. Valid row indices are 0...r-1. Mark a cell visited **when it is pushed**. If cells are marked when popped, the same cell can be pushed many times, which wastes a lot of memory and time.

Bugs: (1) `dy = {0, 0, 1, -1}` (2) `nx < r` (3) mark visited on push, including the start cell

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int r, c; cin >> r >> c;
    vector<string> g(r);
    for (auto &s : g) cin >> s;
    vector<vector<bool>> vis(r, vector<bool>(c, false));
    int dx[] = {1, -1, 0, 0}, dy[] = {0, 0, 1, -1};
    int cnt = 0;
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            if (g[i][j] == '1' && !vis[i][j]) {
                cnt++;
                queue<pair<int,int>> q;
                q.push({i, j});
                vis[i][j] = true;
                while (!q.empty()) {
                    auto [x, y] = q.front(); q.pop();
                    for (int d = 0; d < 4; d++) {
                        int nx = x + dx[d], ny = y + dy[d];
                        if (nx >= 0 && ny >= 0 && nx < r && ny < c && g[nx][ny] == '1' &&
!vis[nx][ny]) {
                            vis[nx][ny] = true;
                            q.push({nx, ny});
                        }
                    }
                }
            }
    cout << cnt << endl;
}

```

BH2. Longest Unique-Character Key (Java)

Concept explained: A sliding window [left, i] keeps the characters unique. `last[c]` stores where each character was last seen and must start at `-1`, meaning "never seen". Java arrays start at 0, which is a real index. When a character repeats, left jumps past its previous position, but **never moves backwards**, which is why `Math.max` is needed (try "abba"). The window length is `i - left + 1`.

Bugs: (1) missing `Arrays.fill(last, -1)` (2) `left = Math.max(left, last[c] + 1)` (3) `i - left + 1`

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        String s = new Scanner(System.in).next();
        int[] last = new int[128];
        Arrays.fill(last, -1);
        int left = 0, best = 0;
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (last[c] >= 0) left = Math.max(left, last[c] + 1);
            last[c] = i;
            best = Math.max(best, i - left + 1);
        }
        System.out.println(best);
    }
}
```

BH3. Resource Allocation (0/1 Knapsack) (Python)

Concept explained: `dp[c]` is the best value with capacity `c`, so it needs indices `0...W` (size `W+1`). In **0/1** knapsack each item may be used once. Looping capacity **downwards** makes sure `dp[c-w]` still holds the value from before this item was considered. Looping upwards would let the same item be reused, which is the unbounded knapsack. The input order is (weight, value).

Bugs: (1) `[0] * (W + 1)` (2) unpack as `for w, v in items` (3) `range(W, w - 1, -1)`

```
n, W = map(int, input().split())
items = [tuple(map(int, input().split())) for _ in range(n)]
dp = [0] * (W + 1)
for w, v in items:
    for c in range(W, w - 1, -1):
        dp[c] = max(dp[c], dp[c - w] + v)
print(dp[W])
```